

COMP7503 Multimedia Technologies

Fundamentals of Compression

Dr. Bill LUO

THE UNIVERSITY OF HONG KONG

DEPARTMENT OF COMPUTER SCIENCE

Need for Compression

- Multimedia data such as images, audio files, and video files takes more space than text-based data
- Video data is currently the single largest network bandwidth and hard disk space consumer
- What would be the uncompressed size of a 90-mins movie in 1080p format?
  - Frame Size = 1920 x 1080 x 3 = 6,220,800 Bytes
  - Data per second = 6,220,800 x 25 = 155.52 MB
  - Data per min = 155.52 MB x 60 = 9.3312 GB
  - Data for 90-mins movie = 839.81 GB
- The size of a Blu-ray disc is ~50 GB only



Black-and-White Image

- Consider the B/W Image as follows, where black pixels are encoded with 1 and white pixels are encoded with 0

```
0000000000000000
0000011111100000
0000100000110000
0000100011010000
0000100110010000
0000111000010000
0000011111100000
0000000000000000
```

- The B/W image is of size 16 x 8, and thus it requires 128 bits to represent it
- If we represent this B/W image the number of consecutive zeros and ones starting with the zeros
  - There representation becomes 21 6 9 1 5 2 8 1 3 2 1 1 8 1 2 2 2 1 8 6 21
  - The highest number represented is twenty-one, we could represent each number by 5 bits
  - The number of bits to represent the same image is 5 x 21 = 105 bits
    - Saving 23 bits



Another Example

- Consider a string
  - RRRRRGGGBBBBBRRRRGB
- Using the same concept, we can represent this string by
  - 5R3G5B4R1G1B
- This method is called *run-length encoding*, or RLE
  - It is used in many image formats, such as BMP, TIFF
  - Do you think RLE is the best way for compression?
  - Can we re-apply RLE on the RLE-compressed data for even more compression?



Information Content

- Level of achievable compression level depends on the amount of information
- Assume we have a 64-bit string consisting of 63 zeros and 1 one, where the one can be at any place in the string
- Now, if we read from left to right, symbol by symbol, until we find the one
- For each symbol b_i at location i, what is the probability that the symbol will be a one (i.e. $P(b_i = 1)$) or zero (i.e. $P(b_i = 0)$)?

i	**1**	**2**	**3**	**4**	**....**	**32**	**33**	**...**	**62**	**63**	**64**
$P(b_i = 0)$	63/64	62/63	61/62	60/61		32/33	31/32		2/3	1/2	0
$P(b_i = 1)$	1/64	1/63	1/62	1/61		1/33	1/32		1/3	1/2	1



Information Content (Cont'd)

- If one appears at the 6-th place, i.e. 000001….
  - What can you say about the symbol from the 7-th place onwards?
  - The zeros after a one carries no information
- Actually, the higher the probability of a symbol, the less information it carries
  - This means the *information content* is inversely proportional to the probability of a symbol's occurrence
  - Information content is defined as $h(x) = \log_2 \frac{1}{p(x)}$
- Revisiting the 64-bit string example again, the table for information content would be

i	**1**	**2**	**3**	**4**	**....**	**32**	**33**	**...**	**62**	**63**	**64**
b_i=0	0.0227	0.023	0.0235	0.0238		0.444	0.458		0.585	1	0
b_i=1	6	5.977	5.954	5.9307		5	5.0666		1.585	1	0



Information Content (Cont'd)

i	1	2	3	4	....	32	33	...	62	63	64
b_i=0	0.0227	0.023	0.0235	0.0238		0.444	0.458		0.585	1	0
b_i=1	6	5.977	5.954	5.9307		5	5.0666		1.585	1	0

- Now, suppose the bit string is 000100……
- What is the sum of information content?
  - Sum = 0.0227 + 0.023 + 0.0235 + 5.9307 = 6
- The general formula

$$\sum_{n=1}^{i} h(b_n) = \left(\sum_{m=66-i}^{64} \log_2 \frac{m}{m-1} + \log_2 \frac{1}{1/(65-i)} \right) = \log_2 \left(\left(\prod_{m=66-i}^{64} \frac{m}{m-1} \right) \times (65-i) \right) = 6$$

- The summation of the information content for each symbol reveals the string's total information content.
  - We need a minimum of six bits to represent the string as described.
  - All we need to save is the index i (a number between zero and sixty-three), which can be represented by a six-digit binary number.



Entropy

- We can represent 64-bits strings by using 6-bits because we know the special structure of the string
- What about encoding other messages, where the message might not be known in advance?
- We define the message that is built based on a language, which consists of a set of symbols or words
  - Each word or symbol has an associated probability
- The expected length of a message X given a set of symbols and their probabilities

$$H(X) = \sum_i P(x_i) h(x_i)$$

- This measure, is also called entropy, which represents the expected information content



Entropy (Cont'd)

- Shannon's source coding theorem gives the information theoretic background for entropy
  - It shows that entropy defines a lower bound for the number of bits that a system can use to encode a message in that alphabet without losing information
- Entropy's value is maximized when the alphabet is uniformly distributed
  - i.e. each character has the same probability
- Given a subset of a string, there is no possibility of a prediction about the next symbols because all symbols have equal probability. Random noise has this property and is therefore incompressible.



Compression Algorithms

- Information content and entropy give us a tool to measure a message's expected minimum length
  - But how do you construct a code for that string that uses the minimal number of bits?
- Take our courses as example
  - Our course code is COMPABCD, where ABCD is a four digit number
  - How many bits are required to represent our course code?
  - $2^{13} = 8192$ and $2^{14} = 16384$, that means it require 14 bits to represent the course code
  - But if we further know that MSc courses have to be COMP7ABC, then, 10 bits is enough for differentiating each course
- To maximize the compression level, we need to find a proper representation scheme for the underlying information



Desirable Encoding Properties

- The code must be unambiguously decodable
  - One coded message corresponds to exactly one decoded message
- The code should be easily decodable
  - Able to find symbol endings and the end of message easily
  - Able to decode online, that is as symbol comes in, it can be decoded without knowing the entire coded message
- The code should be compact



Huffman Coding

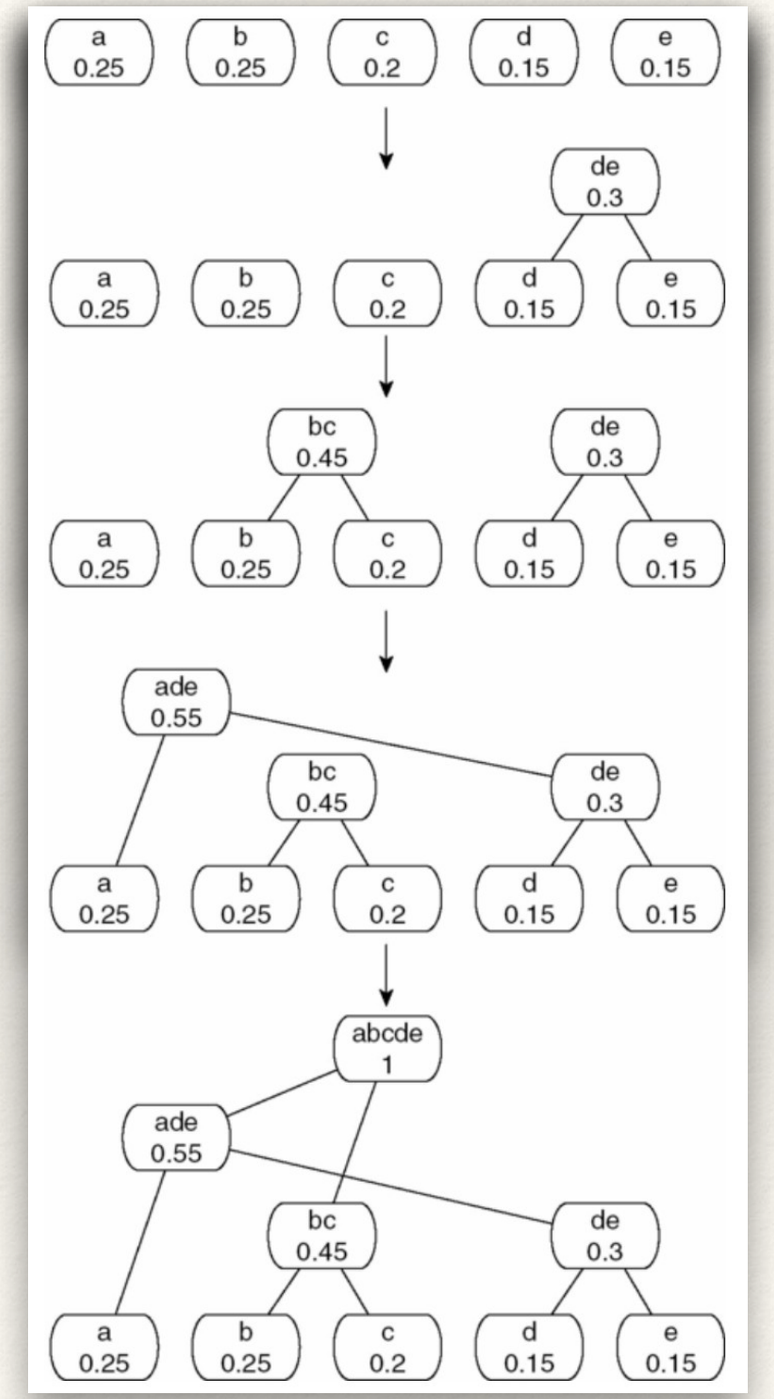
- In 1952, David A. Huffman invented a coding algorithm that produces a code that fulfill the desirable encoding properties stated in the previous slide

- Consider a message that is constructed as a string, where each character could be one of the five symbols *a*, *b*, *c*, *d*, and *e*
  - If we use fixed length coding scheme, how many bits is required to differentiate the five symbols?
  - How many bits is required to represent the message ‘cab’? If we employ fixed length coding scheme?



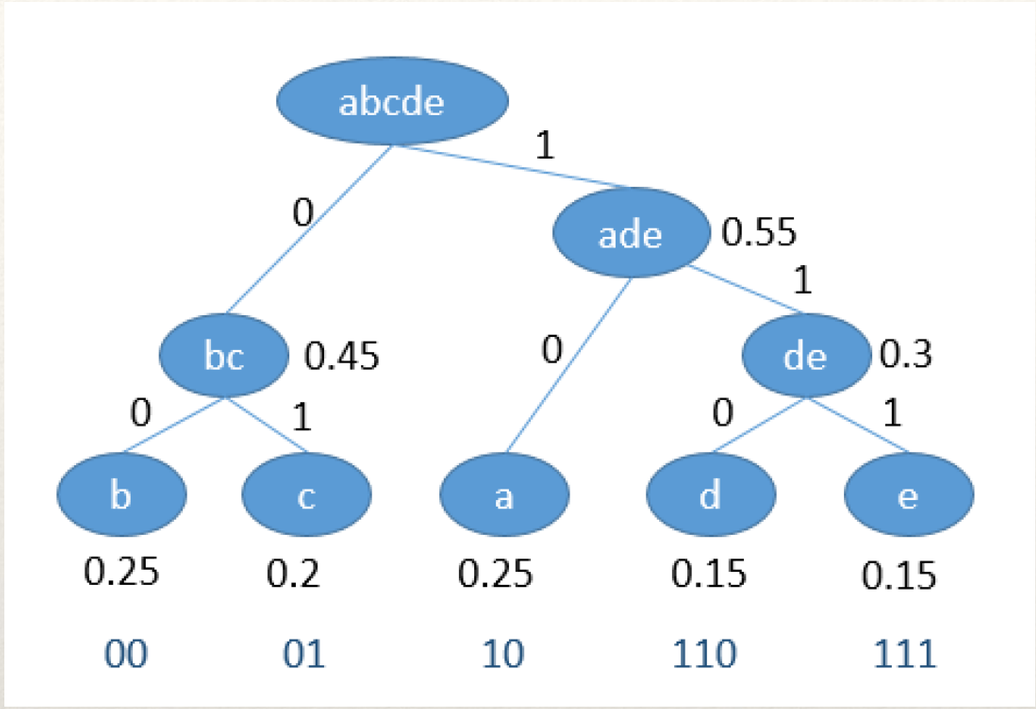
Huffman Coding (Cont'd)

- Huffman algorithm constructs a binary tree in which the symbols are the leaves
  - The leaf nodes containing the frequencies of the symbol they represent are unconnected.
  - A new node whose children are the two nodes with the smallest probabilities, such that the new node's probability equals the sum of the children's probability. The new node is treated as a regular symbol node (a.k.a a compound symbol node)
- The procedure is repeated until only one node remains – the root of the Huffman tree.
- This procedure is also known as source reduction



Huffman Coding (Cont'd)

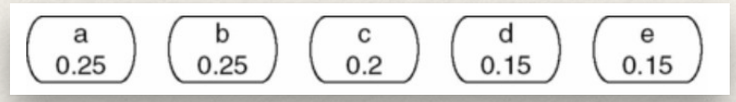
- After the tree is created
  - 0 and 1 are then used to label each left and right branches, respectively
  - This process is called code assignments
- The path from the root to a leaf reveals the code for the symbol.
  - When turning right on a node, a 1 is added to the code;
  - when turning left, a 0 is added. The idea is to have long paths for symbols that occur infrequently and short paths for symbols that occur frequently.
- What would be the code for each symbols?
- How many bits is required to represent the message 'cab'
- Is there any compression achieved when compared with fixed length code?
- Any insights for code construction?
- Can you decode the following message?
  - 11011110110



'cab' can be represented as

01	10	00

Compression Ratio = 9/6 = 1.5



Huffman Coding (Cont'd)

- The Huffman Code itself is
  - A block code: each source symbol is mapped into a fixed sequence of code symbols
  - Instantaneous decodable : each code word in a string of code symbols can be decoded without referencing succeeding symbols
  - Uniquely decodable: any string of code symbols can be decoded in only one way by examining the individual symbols of the string in a left to right manner
- Huffman Code is a common form of Variable Length Coding
  - Philosophy
    - Assign shorter code word to higher probability symbol, and vice versa



Pseudo Code for Huffman Tree Construction

```
// Input: A list W of n frequencies for the symbols
// Output: A binary tree T with weights taken from W
Huffman(W, n):
    Create list F with single-element trees formed from elements of W
    WHILE (F has more than one element)
        Find T1, T2 in F with minimum root values
        Create new tree T by creating a new node with root value T1+T2
        Make T1 and T2 children of T
        Add T to F
Huffman := tree stored in F
```



Pseudo Code for Huffman Tree Encoding/Decoding

```
// Pseudo Code for Huffman Encoding
// Input: A Huffman tree F and a string S containing only symbols in F
// Output: A bit-sequence B
Encode_Huffman(F, S)
    Traverse F completely and create a hash table H
    containing tuples (c,p)
    // c = character, p = path from root to of F to character
    B := []
    FOREACH (symbol s in S)
        Add path H(s) to bit sequence B.
Encode_Huffman := B
```

```
// Pseudo Code for Huffman Decoding
// Input: A Huffman tree F and a bit sequence B
// Output: An uncompressed string S
Decode_Huffman(F, B)
    Traverse F completely and create a hash table H
    containing tuples (p, c)
    // c = character, p = path from root to of F to character
    S := ""
    node n = root node of F
    FOREACH (bit b in B)
        if (b==0) n := left children of n
        if (b==1) n := right children of n
        if (n has no children)
            Add character represented by n to S
            n := root node of F
Decode_Huffman := S
```



Truncated Huffman Coding

- A truncated Huffman code is generated by Huffman coding only the *k* most probable symbols of the source
- A prefix code followed by a suitable fixed-length code is used to represent all the other symbol
- The prefix code can be generated as the k+1th Huffman code word, whose probability is the sum of the probabilities of all the symbols other than the *k* most probable ones
- This approach reduces the no. of source reductions and code assignments needed, and produces a near optimal variable length code



Truncated Huffman Coding (Cont'd)

Original source			Source reduction							
Sym.	Prob.	Code	1		2		3		4	
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0
a_6'	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1
a_1	0.1	011	0.1	011	0.2	010	0.3	01		
a_4	0.1	0100	0.1	0100	0.1	011				
a_3	0.06	01010	0.1	0101						
a_5	0.04	01011								

a_6	0.04	00 0000
a_7	0.04	00 0001
a_8	0.04	00 0010
a_9	0.03	00 0011
a_{10}	0.03	00 0100
a_{11}	0.03	00 0101
a_{12}	0.03	00 0110
a_{13}	0.03	00 0111
a_{14}	0.03	00 1000

- Can you work out the theoretical optimal bits/symbol?
- Can you work out the average bits/symbol based on the Truncated Huffman Coding here?

- Theoretical optimal bits/symbol = 3.0903 bits/symbols
- Average bits/symbol based on the Truncated Huffman Coding = 3.4 bits/symbol



Shift Coding

- A shift code is generated by
  - Arranging the source symbols so that their probabilities are monotonic decreasing
  - Dividing the total no. of symbols into symbol blocks of equal size
  - Coding the individual elements within all block identically
  - Adding special ***shift-up*** and/or ***shift-down*** symbols to identify each block



Shift Coding (Cont'd)

Source symbol	Probability	Binary Code	Huffman	Truncated Huffman	Binary Shift	Huffman Shift
Block 1						
a_1	0.2	00000	10	11	000	10
a_2	0.1	00001	110	011	001	111
a_3	0.1	00010	111	0000	010	110
a_4	0.06	00011	0101	0101	011	0111
a_5	0.05	00100	00000	00010	100	0110
a_6	0.05	00101	00001	00011	101	0101
a_7	0.05	00110	00010	00100	110	0100
Block 2						
a_8	0.04	00111	00011	00101	111 000	00 10
a_9	0.04	01000	00110	00110	111 001	00 111
a_{10}	0.04	01001	00111	00111	111 010	00 110
a_{11}	0.04	01010	00100	01000	111 011	00 0111
a_{12}	0.03	01011	01001	01001	111 100	00 0110
a_{13}	0.03	01100	01110	100000	111 101	00 0101
a_{14}	0.03	01101	01111	100001	111 110	00 0100
Block 3						
a_{15}	0.03	01110	01100	100010	111 111 000	00 00 10
a_{16}	0.02	01111	010000	100011	111 111 001	00 00 111
a_{17}	0.02	10000	010001	100100	111 111 010	00 00 110
a_{18}	0.02	10001	001010	100101	111 111 011	00 00 0111
a_{19}	0.02	10010	001011	100110	111 111 100	00 00 0110
a_{20}	0.02	10011	011010	100111	111 111 101	00 00 0101
a_{21}	0.01	10100	011011	101000	111 111 110	00 00 0100
Entropy	4.0					
Average length		5.0	4.05	4.24	4.59	4.13

Lempel-Ziv Algorithm

- The LZ (named derived from its creators - Abraham ***Lempel*** and Jacob ***Ziv***) family of algorithms
  - LZ77 and LZ78 algorithms were respectively developed in 1977 and 1978
  - LZ77 and LZ78 use completely different approaches, despite similarity in their names
- This family algorithms is best suited for limited alphabets, such as text and/or source code
  - Popular image formats, GIF and PNG, use one variant of each of the algorithms when only a limited amount of color is used
- Conceptual idea of LZ77
  - LZ77 achieves compression by replacing portions of the data with references to matching data that have already passed through the encoder and decoder
    - It works on past data



LZ77

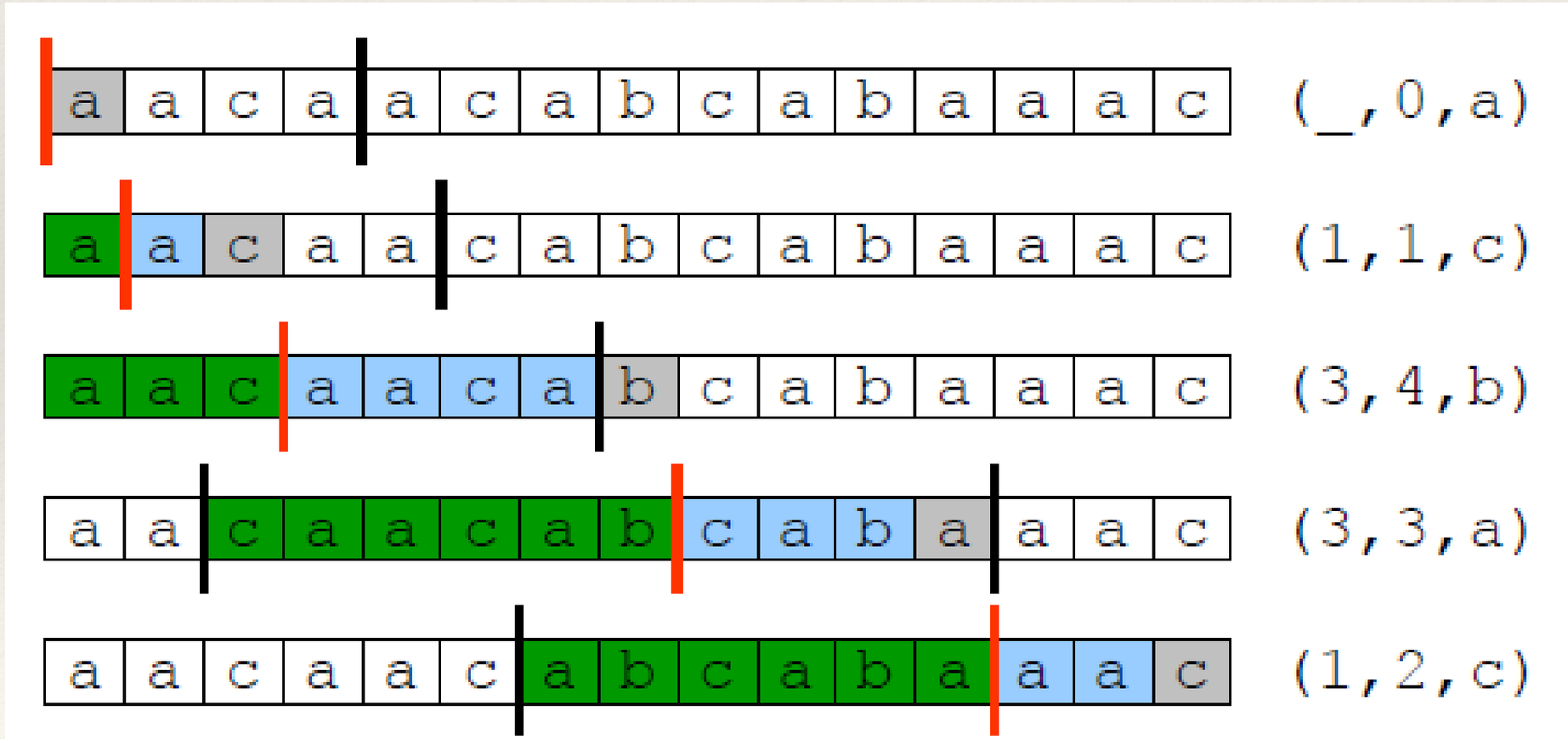
- This algorithm works based on
  - A fixed number of symbols - the ***look-ahead-buffer*** (LAB) - to be coded; and
  - A fixed number of symbols from the past - the ***search buffer*** (SB)
- It encode the symbols in LAB, the algorithm searches the SB backward for the best match
- Then encoding (ofter referred to as ***token***) is encoded by a tuple (D, L)
  - From laymen perspective, a token says "each of the next L symbols is equal to the D symbols behind it in the uncompressed stream."
  - The distance D is often referred to as offset
- The SB+LAB is often called a sliding window
  - Typical sizes for sliding window are several kilobytes (2 kB, 4kB, 32kB, etc.)



LZ77 (Cont'd)

- Encoding of the string "aacaacabcabaaac" using LZ77 compression with LAB=4 and SB=6 (D, L, C)

  D: relative distance before 'red' bar (distance between match symbols of the past)
  L: the length of 'blue' word (best match)
  C: the grey character (next non-matched character)



Pseudo-code for an LZ77 encoder/decoder

```
// LZ77 encoder
// Input: A lookaheadbuffer LAB a substring of a message...
// Output: An LZ77 encoding C
Encode_LZ77(LAB)
    C := ""
    WHILE (LAB not empty)
      p := position of the longest match in the window for the LAB
      l := length of the longest match
      Add the tuple (p, l) to C
      Add the first character in LAB to C
      Shift LAB by l
Encode_LZ77 := C

// LZ77 decoder
// Input: A string C containing LZ77 code
// Output: A string S containing the decoded code
Decode_LZ77(C)
 S := ""

 FOREACH (token in C)
   Read token and obtain the triple (p, l, c)
   // p = position, l = length, c = character

   Add to S the l characters from position length(S)-p
   Add to S the character c
Decode_LZ77 := S
```



LZ77 Encoder Example in C

```
#include <iostream>
using namespace std;


typedef struct _Tuple {
     int p ;
     int l ;
     char c ;
     struct _Tuple *next ;
} Tuple ;


typedef struct {
     char *data ;
     int len ;
     int curr ;
     int sb_len ;
     int lab_len ;
} LAB ;


int matchString(char *a, char *b, int
len_a) {
     int matchedCount = 0;

     while(a[matchedCount] ==
b[matchedCount] && matchedCount <
len_a) {
          matchedCount++ ;
     }
     return matchedCount ;
}
```

```
Tuple longestMatchedString(LAB *lab) {
     int longestLen = 0 ;
     int matchedLen = 0 ;
     int position = 0 ;
     Tuple C ;

     int min, max ;

     min = lab->curr - lab->sb_len ;
     if(min < 0) min = 0 ;
     max = lab->curr + lab->lab_len - 1;
     if(max >= lab->len) max = lab->len - 1 ;

     int i ;

          for(i = min; i < lab->curr; i++) {
               matchedLen = matchString(lab->data + lab->curr, lab->data + i,
lab->len) ;
                    if(matchedLen > longestLen) {
                         longestLen = matchedLen ;
                         position = lab->curr - i ;
                    }
          }

     C.p = position ;
     C.l = longestLen ;
     lab->curr += longestLen ;
     C.c = lab->data[lab->curr];
     C.next = NULL ;

     return C ;
}
```



LZ77 Encoder Example in C

```
Tuple *Encode_LZ77(LAB lab) {

	Tuple *C = new Tuple ;
	Tuple *CPtr = NULL ;

	CPtr = C ;
	while(lab.curr < lab.len) {
		*CPtr = longestMatchedString(&lab) ;
		lab.curr++ ;
		if(lab.curr < lab.len) {
			CPtr->next = new Tuple ;
			CPtr = CPtr->next ;
		} else {
			CPtr->next = NULL ;
		}
	}

	return C ;
}

void Decode_LZ77(Tuple *C, LAB *lab) {

	Tuple *Cnext ;

	Cnext = C ;
	while(Cnext) {
		for(int i = 0; i < Cnext->l; i++) {
			lab->data[lab->curr]=lab->data[lab->curr - Cnext->p] ;
			lab->curr++ ;
		}
		lab->data[lab->curr] = Cnext->c ;
		lab->curr++ ;
		Cnext = Cnext->next ;
	}
}

int main(int argc, char *argv[]) {

	LAB lab ;
	Tuple *C, *Cnext ;
	char defaultMessage[] = "cabbdcbaacbdddabda" ;

	if(argc <= 1) {
		lab.data = defaultMessage ;
	} else {
		lab.data = argv[1] ;
	}

	lab.sb_len = 7 ;
	lab.lab_len = 3 ;
	lab.curr = 0 ;
	lab.len = strlen(lab.data);

	printf("Encoding Message %s\n", lab.data);

	C = Encode_LZ77(lab) ;

	Cnext = C ;
	while(Cnext) {
		printf("(%d, %d, %c)\n", Cnext->p, Cnext->l, Cnext->c) ;
		Cnext = Cnext->next ;
	}

	lab.sb_len = 7 ;
	lab.lab_len = 3 ;
	lab.curr = 0 ;
	lab.len = 0 ;
	lab.data[0] = 0 ;

	Decode_LZ77(C, &lab) ;

	while(C) {
		Cnext = C->next ;
		delete C ;
		C = Cnext ;
	}

	printf("Decoded Message %s\n", lab.data);

	return 0;
}
```



LZ77 Encoder in C vs Pseudo Code

```
Tuple *Encode_LZ77(LAB lab) {

      Tuple *C = new Tuple ;
      Tuple *CPtr = NULL ;

      CPtr = C ;
      while(lab.curr < lab.len) {
            *CPtr = longestMatchedString(&lab) ;
            lab.curr++ ;
            if(lab.curr < lab.len) {
                  CPtr->next = new Tuple ;
                  CPtr = CPtr->next ;
            } else {
                  CPtr->next = NULL ;
            }
      }

      return C ;
}

void Decode_LZ77(Tuple *C, LAB *lab) {

      Tuple *Cnext ;

      Cnext = C ;
      while(Cnext) {
            for(int i = 0; i < Cnext->l; i++) {
                  lab->data[lab->curr]=lab->data[lab->curr - Cnext->p] ;
                  lab->curr++ ;
            }
            lab->data[lab->curr] = Cnext->c ;
            lab->curr++ ;
            Cnext = Cnext->next ;
      }
}
```

```
// LZ77 encoder
// Input: A lookaheadbuffer LAB a substring of a message...
// Output: An LZ77 encoding C
Encode_LZ77(LAB)
   C := ""
   WHILE (LAB not empty)
     p := position of the longest match in the window for the LAB
     l := length of the longest match
     Add the tuple (p, l) to C
     Add the first character in LAB to C
     Shift LAB by l
 Encode_LZ77 := C


// LZ77 decoder
// Input: A string C containing LZ77 code
// Output: A string S containing the decoded code
Decode_LZ77(C)
 S := ""

 FOREACH (token in C)
   Read token and obtain the triple (p, l, c)
   // p = position, l = length, c = character

   Add to S the l characters from position length(S)-p
   Add to S the character c
Decode_LZ77 := S
```



Variants of LZ77

- LZR
  - Unrestricted search buffer
  - Variable bit-length tokens
- LZH
  - The token values are compressed using a Huffman encoding
- The currently most popular LZ77-based compression is called ***DEFLATE***
  - It is part of the very common Unix compression program "gzip"



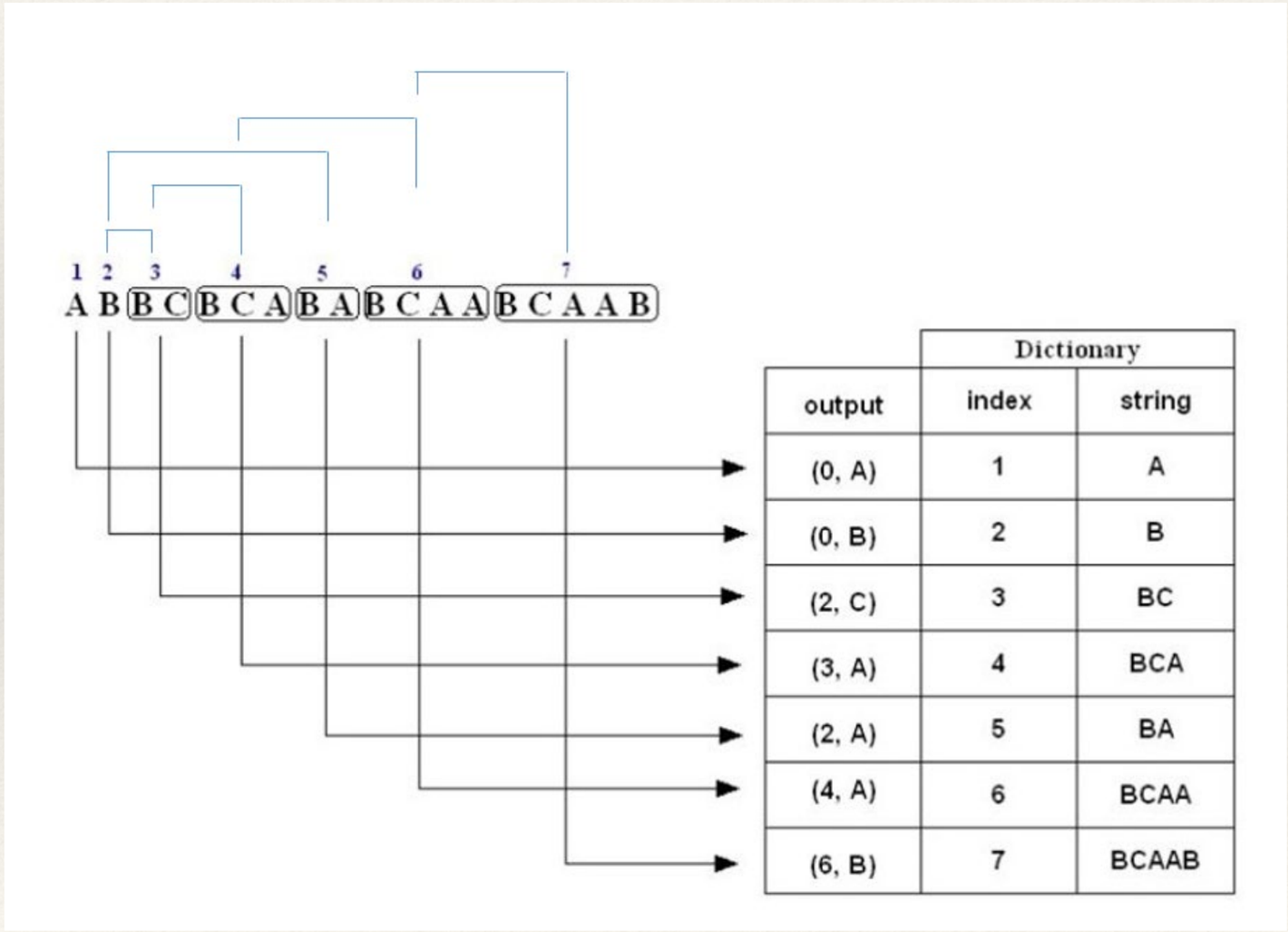
LZ78

- LZ78 algorithms achieve compression by replacing repeated occurrences of data with references to a ***dictionary*** that is built based on the input data stream
- The algorithm scans the input buffer for the longest match of the buffer with a dictionary entry until it can no longer find a match
  - It then output the following
    - Location of the word in the dictionary
    - The character that caused a match failure
  - And, it adds the resulting word to the dictionary



LZ78 (Cont'd)

- Encoding of the string "ABBCBCABBCAABCAAB" using LZ78



	Dictionary	
output	index	string
(0, A)	1	A
(0, B)	2	B
(2, C)	3	BC
(3, A)	4	BCA
(2, A)	5	BA
(4, A)	6	BCAA
(6, B)	7	BCAAB

Pseudo-code for an LZ78 encoder/decoder

```
// LZ78 encoder
// Input: A string S
// Output: A string C containing the LZ78 code
Encode_LZ78(S)
    Start with empty dictionary D
    C := ""
    prefix := "" // stores the prefixes
    FOREACH (character c in S)
        IF (prefix + c is in D)
            prefix := prefix + c
        IF (prefix + c is not in D)
            Add the string prefix + c to D
            Add the index of prefix in D to C
            Add c to C
            prefix := ""
Encode_LZ78 := C
```

```
// LZ78 decoder
// Input: An LZ78 encoded string C
// Output: A string S containing the original message
Decode_LZ78(C)
    Start with empty dictionary D
    old index := index value of the first token in C
    S := ""
    FOREACH (token t in C)
        index := index value of t
        IF (D has entry with index)
        Add the string at index to S
         c := first character of the string at index
        Concatenate entry at old index and c and add to D
        IF (D has no entry with index)
        c := first character of the string at old index
        Concatenate entry at old index and c and add to D
        Concatenate entry old index with c and add to S
        Old index := index
Decode_LZ78 := S
```



Variants of LZ78

- In the original LZ78 algorithm, the dictionary's size is unrestricted
  - The index values must be saved with a variable number of bits
  - The algorithm uses the dictionary's size to determine the index's bit length
    - Number of bits required = $\log_2(i)$ where i is the largest dictionary index
- LZC
  - It uses a maximum size for the dictionary
  - If it reaches the maximum number of entries, the algorithm continues with the current dictionary under the hope that the output file does not become too long
  - IF the compression ratio starts to worsen, the dictionary is cleared
    - i.e. the compression algorithm is restarted
- LZW
  - Dictionary starts with all possible input symbols as first entries
    - vs LZ78 starts with an empty dictionary
  - A token pair (index, character) is required in LZ78, but in LZW, only the dictionary index is transmitted



LZW

- Encoding of the string "abacbabaccbabbaca" using LZW

Step	Input	Encoded Output	New Dictionary Entry/Index
1	a		
2	b	0 (a)	ab / 3
3	a	1 (b)	ba / 4
4	c	0 (a)	ac / 5
5	b	2 (c)	cb / 6
6	a		
7	b	4 (ba)	bab / 7
8	a		
9	c	4(ba)	bac / 8
10	c	2 (c)	cc / 9
11	b		
12	a	6 (cb)	cba / 10
13	b		
14	b	3 (ab)	abb / 11
15	a		
16	c		
17	a	8 (bac)	baca / 12
18		0 (a)	

Index	Dictionary Entry
0	a
1	b
2	c
3	ab
4	ba
5	ac
6	cb
7	bab
8	bac
9	cc
10	cba
11	abb
12	baca



Arithmetic Coding

- Huffman encoding's limitation
  - The messages can only be encoded using an integer number of bits per symbol
- Consider the following symbols with the following probability

x_i	A	B	c	#
$P(x_i)$	0.5	0.2	0.2	0.1

  - What would be the theoretical minimum number of bits to represent the symbols?

$$H(X) = \sum_i P(x_i) h(x_i) = \sum_i P(x_i) \log_2 \frac{1}{P(x_i)} = -\sum_i P(x_i) \log_2 P(x_i)$$

$$= -0.5 \log_2 0.5 - 0.2 \log_2 0.2 - 0.2 \log_2 0.2 - 0.1 \log_2 0.1$$

$$= 0.5 + 0.4644 + 0.4644 + 0.3322 \approx 1.76 \ bits$$

  - Huffman coding cannot generate 1.76 bits per symbol
- Arithmetic Coding was designed to address this issue



Arithmetic Coding (Cont'd)

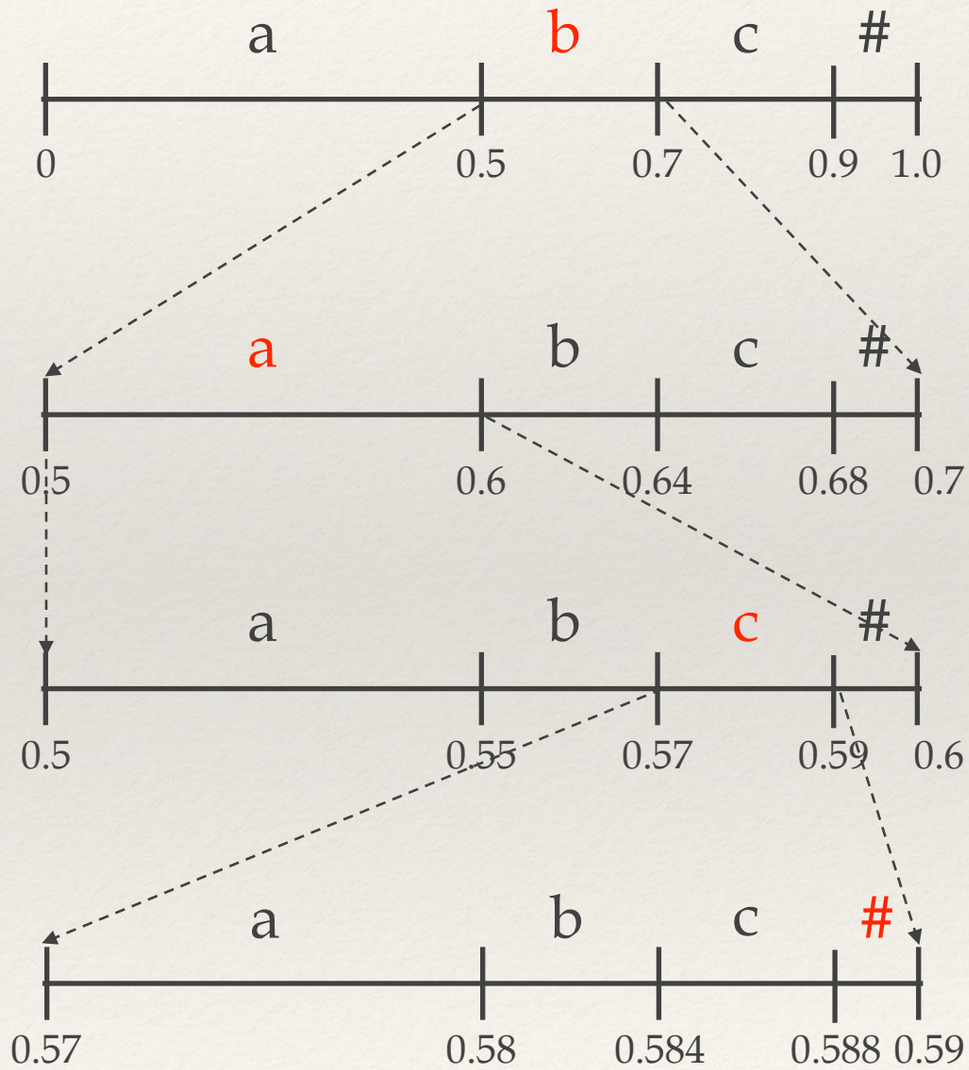
- Unlike the variable-length codes described previously, Arithmetic Coding generates *nonblock* codes
  - A one-to-one correspondence between source symbols and code words does not exist
- An entire sequence of source symbols (or message) is assigned a single arithmetic code word
  - The code word itself defines an interval or real numbers between 0 and 1
  - As the number of symbols in the message increases, the interval used to represent it becomes smaller and smaller
    - Each symbol of the message reduces the size of the interval in accordance with its probability of occurrence
    - The number of information units (say, bits) required to represent the interval becomes larger
  - A termination symbol is usually required



Arithmetic Coding (Cont'd)

- Encoding of the message "bac#" by Arithmetic Coding = 0.58984375_{10} or 0.10010111_2

x_i	A	B	c	# Termination Symbol
$P(x_i)$	0.5	0.2	0.2	0.1
Step 1: read "b"				
Partition (decimal)	[0,0.5)	[0.5,0.7)	[0.7,0.9)	[0.9,1)
Partition (binary)	$[0,0.1)_2$	$[0.1,0.\overline{10110})_2$	$[0.\overline{10110},0.\overline{11100})_2$	$[0.\overline{11100},1)_2$
Step 2: read "a"				
Partition (decimal)	0.5,0.6)	[0.6,0.64)	0.64,0.68)	[0.68,0.7)
Partition (binary)	$[0.1,0.\overline{1001})_2$	$0.\overline{1001}, 0.1010001 ...)_2$	$[0.1010001..., 0,1010111 ...)_2$	$[0.1010111...,0.\overline{10110})_2$
Step 3: read "c"				
Partition (decimal)	[0.5,0.55)	[0.55,0.57)	[0.57,0.59) $\begin{bmatrix}0.100100011...,\\0.100101110...\end{bmatrix}_2$	[0.59,0.6)
Step 4: read #				
Partition (decimal)	[0.57,0.58)	[0.58,0.584)	[0.584,0.588)	[0.588,0.59) $=\begin{bmatrix}0.100101101...,\\0.100101110...\end{bmatrix}_2$



Pseudocode for Arithmetic Coding

```
// Arithmetic Coding Encode
// Input: A string S containing a message, ending with a stop symbol
// Output: An arithmetically encoded string C,
// a table P mapping probability ranges to symbols
Encode_Arith(S)
    FOREACH (character c in S)
        Get the frequency and assume as probability p[c]
    Create a table P that assigns characters to probability ranges in [0,1), each range with a size proportional to p[c]
    lower_bound := 0
    upper_bound := 1
    FOREACH (character c in S)
        current_range := upper_bound-lower_bound
        upper_bound := lower_bound+(current_range*upper_bound[c])
        lower_bound := lower_bound+(current_range*lower bound[c])
    C=upper_bound+lower_bound/2.0
Encode_Arith := (C, P)
```

```
// Arithmetic Coding Decode
// Input: An arithmetically encoded string C,
// a table P mapping probability ranges to symbols
// Output: A string S containing the original message
Decode_Arith(C, P)
    encoded_value := C
    WHILE (we have not seen the terminal symbol)
        s := symbol in P where encoded_value is within its range
        //remove effects of s from encoded_value
        current_range = upper_bound of c – lower bound of c
        encoded_value = (encoded_value-lower_bound of c)/current_range
    Add s to S
Decode_Arith := S
```



Technical Challenge for Arithmetic Coding

- Precision problem
  - Accurate representation of interval boundaries may not be possible due to limited number of bits in representing numbers
    - 32-bit or 64 bit floating point numbers
- Whenever the range decreases to the point at which the interval's start and end values share certain beginning digits, the coder sends those digits to the output
  - Then scale up the number, and continue to add new digits to the scaled number



Weakness of Entropy-based Compression Methods for Multimedia Data

- The compression techniques we have covered thus far try to reconstruct the data in full
  - Lossless compression methods
- The compression obtained based on these entropy compression routines is usually a factor of two or less because
  - Entropy is usually higher for multimedia signals than for text data
  - Sampled signals contain noise, the possibility for finding ideal repeating patterns is low
  - Multimedia data is usually multidimensional. To explore redundancies, understanding of data's basic structure is required

